CS312 Final Term Handouts Highlighted By VU-Gateway

Imp Mcqs + Question Highlighted

Join VU-Gateway Whatsapp Group For Imp highlighted Handouts, Notes, Files , Notification and preparation:

https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

VU-Gateway

Module 12: Anomalies

A. What is Anomaly?

Anomalies can be defined as the problems that can occur due to poor planning and designing of the databases. This usually occurs when a single table is created instead of creating multiple tables. An anomaly is an irregularity, or something which deviates from the expected or normal state.

PPPPP. Why Anomalies are Critical

As mentioned above, anomalies are critical as it can affect the end user experience. Besides, duplication or redundancy in the data is increased due to anomalies which in results create other serious issues. Anomalies may not have a huge impact on the small databases but a database with a large data size may have serious performance issues caused by anomalies. There are three types of anomalies which are explained below.

QQQQQ. Insert Anomaly

Insert anomaly occurs when certain data cannot be entered without entering unwanted data. In other words, in other words, some attributes cannot be entered without the presence of some other attributes. This results from poor design decision of storing data in just one table instead of multiple tables. As in the example below, if we want to enter a new product (pid, pname, price) then we have to add a customer (cid) against the product which might not be possible at the time of entering the product meaning to add a wanted row (product) we have to enter an unwanted (customer) row because CID is primary key and it can't be left null.

		Product			
Pid	**Pname**	**Price**	**CID**	**Cname**	**Address**
1	HD	1500	1	Ahmed	12-K, DHA
2	RAM	2500	1	Ahmed	12-K, DHA
3	MotherBoard	12000	3	Kashif	M/2-Cavalry
4	Mouse	400	2	Ali	183//A-1, Gulberg
5	LCD	20000	3	Kashif	M/2-Cavalry

VU Gateway

RRRRR. Delete Anomaly

This anomaly occurs when the data is lost due to the deletion of some other data. The reasons of this anomaly are same as mentioned above i.e. poor design decision. Due to this anomaly certain attributes are lost due to the deletion of other attributes. If we want to delete pid : 4 then we will be deleting cid=2 also because when we delete complete row is deleted and there is only record for cid=2 meaning we might end up losing customer data when we delete product data which is a loss of unwanted data.

Figure 7: Delete Anomaly

Product

Pid	Pname	Price	CID	Cname	Address
1	HD	1500	1	Ahmed	12-K, DHA
2	RAM	2500	1	Ahmed	12-K, DHA
3	MotherBoard	12000	3	Kashif	M/2-Cavalry
4	Mouse	400	2	Ali	183//A-1, Gulberg
5	LCD	20000	3	Kashif	M/2-Cavalry

SSSSS. Update Anomaly

An Update Anomaly exists when one or more instances of duplicated data are updated, but not all. An update anomaly occurs when the same data item has to be updated more than once. This can lead to errors and inconsistency of data. The same information can be expressed on multiple rows; therefore all the instances must be updates. If this is not done properly, update anomaly arises. If there are multiple records and we fail to update all the instances then it will result in data-inconsistency across database which will result in wrong reporting of data which is not acceptable.

Product

Pid	Pname	Price	CID	Cname	Address
1	HD	1500	1	Ahmed	12-K, DHA
2	RAM	2500	1	Ahmed	12-K, DHA
3	MotherBoard	12000	3	Mohsin	M/2-Cavalry
4	Mouse	400	2	Ali	183//A-1, Gulberg
5	LCD	20000	3	Kashif	M/2-Cavalry

In the table given above if we decide to update Cname of cid=3 and we fail to update all the rows (cname) then it will end up in wrong information. Records with yellow color are showing that cid=3 is having different Cname and system will report you both Cname against CID=3 and its wrong. It is result that all the records are not updated.

VU-Gateway

Module 13: Normalization

A. Normalization Basics

Database normalization is the process of storing the data efficiently in order to reduce data redundancy and undesirable characteristics from the system. The two main objectives of the database normalization are to reduce redundancy and to make sure the relationships / dependencies are logical. That's why the related data is stored together. This process serves as a solution to database anomalies as normalization is a method to remove all these anomalies and bring the database to a consistent state.

TTTTT. Normalization through Reports

Normalization can be done through physical data model or through reports. In the course we will discuss Normalization through reports primarily due the reason that it will give a clear structure of the whole process.

The process of normalization consists of three steps called Normal Forms and these are as under:

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)

UUUUU. First Normal Form (1NF)

First normal form (1NF) sets the very basic rules for an organized database: As per First Normal Form, no two Rows of data must contain repeating group of information i.e. each set of column must have a unique value, such that multiple columns cannot be used to fetch the same row. Each table should be organized into rows, and each row should have a primary key that distinguishes it as unique. These rules can be summarized as following:

- Data should be organized in tables
- There should be no repeating groups
- There should be Primary Keys in each tables.
- Primary key non-repeating group should be written with every group following the first non-repeating

Student

Sid	Sname	Address	Total_Courses-Reg	Status
1	Ali	12-k	4	Full-time

Course

Cid	Cname	Credithrs
1	DB	3
2	OOP	4
3	Web Eng	4

Semester

Semid	Start Date	End Date
Fall-15	10-Oct-15	26-Feb-16
Spring-16	15-Mar-16	30-Jun-16

VU Gateway

group as FK.

- First group has to be non-repeating group

VVVVV. Definition of Non- Repeating group:

Group in which there is only one row in the given scenario – In the example given above student is a non-repeating group as there is only one row given in the report.

WWWWW. Definition of Repeating group:

Group in which there are more than one row in the given scenario – In the example given above course and semester are non-repeating group as there is only one row given in the report.

L. Example of First Normal Form (1NF)

Based on the figure 24 above, the following 1NF is as below entities and attributes can be defined:

Primary key of first group which has to be Non-repeating group – Student group; has to be written as FK with every other following group – Course and Semester. Just like One-to-Many primary key of Parent table is written as FK in child table, student is a parent table and Course & Semester table are child table.

First Normal Form (1NF)	
Entities	**Attributes**
Student	Sid, Sname, Total_Registered_Courses, Status
Course	Cid, Cname, Credithrs, Sid
Semester	Semid, Sid, Start Date, End Date

The data presented in the above format fulfill all the criteria of being in first normal form.

YYYYY. Second Normal Form (2NF)

2NF is derived from 1NF which means that the prerequisite to perform 2NF is 1NF. This rules states that here must not be any partial dependency of any non- key column on primary key.The objective of the second normal form is to take data that is only partly dependent on the primary key and enters that data into another table.

ZZZZZ. Example of Second Normal Form (2NF)

Considering again the data presented in the figure 24 above, the following conditions needs to be fulfilled:

- First Normal Form (1NF) is achieved
- There should be no partial dependency
- Each non-key attribute should be fully-functional dependent on Key

VU Gateway

First Normal Form (1NF)	
TableName	**Attributes**
Student	Sid, Sname, Total_Registered_Courses, Status
Course	Cid, Cname, Credithrs, Sid
Semester	Semid, Sid, Start Date, End Date

Second Normal Form (2NF)		
TableName	**Attributes**	**Comments**
Student	Sid, Sname, Total_Registered_Courses, Status	No Change. Table with single PK is already in 2NF.
Course	(Cid, Cname, Credithrs, Sid)	Cid ⇒ Cname & Cid ⇒ Credit Hour because to find out correct value of cname &credithr only value of CID is required mean if we don't know the value of SID even then correct value of cname, credithr can found – Non-key attributes (cname, credithrs) are dependent on part of Composite key – only CID ; this is partial dependency, **course table is not in 2NF**
Course	cid, sname, credits	New Table (update of course table) **– In 2NF now**
Student-course	cid, id	Original table – Course is renamed to student_course and cid, sid are mention. **– In 2NF now**
Semester	Semid, Sid, Start Date, End Date	Semid⇒ start date, end date… Both have functional dependency on part of CPK i-e semid only. To find out correct start and end date **only** value of Semid is required.**Semester table is not in 2NF**
Semester_info	(semid, startdate, enddate)	New Table - **– In 2NF now**
Semester	(semid, sid)	Original table - **– In 2NF now**

AAAAAA. Third Normal Form (3NF)

Third Normal form applies that every non-prime attribute of table must be dependent on primary key, or we can say that, there should not be the case that a non-prime attribute is determined by another non-prime attribute. This is known as transitive dependency. So this transitive dependency should be removed from the table and also the table must be in Second Normal form.

BBBBBB. Example of Third Normal Form (3NF)

Continuing with the same example, the third normal form would be like:

Note: First Normal Form (1NF) and Second Normal Form (2NF) are shown above.

VU-Gateway

Third Normal Form (3NF)		
TableName	**Attributes**	**Comments**
Course	(cid, sname, credits)	No Change
Student-course	(cid, id)	No Change
Semester	(semid, sid)	No Change
Semester_info	(semid, startdate, enddate)	No Change
Student	Sid, Sname, Total_Registered_Courses, Status	Total Course Registered ⇒ Status. Both are non key attributes and have transitive dependency because if value of total_course_registered is known then we can tell the value of status – Part-time or full-time student but if value of status of is known then correct value of total_course_registered is not known. For example if total_course_registered is 4 then status is full-time but if status is full-time it is not possible to tell whether total_course_registered is 4 or 5 or 6 , in this cause status is dependent on total_course_registered and both are non-key . This relation is not in 3NF
Student-Status	Total_courses_registered, status	New Table
Student	sid,sname, reg_courses	New Table (update of Student table)

After applying the normalization up to third normal form (3NF) the data would be in the following shape:

TableName	**Attributes**
Course	(cid, sname, credits)
Student-course	(cid, id)
Semester	(semid, sid)
Semester_info	(semid, startdate, enddate)
Student-Status	Total_courses_registered, status
Student	sid,sname, reg_courses

CCCCCC. Second Example of Normalization

Consider the data presented in the figure 25 on right having the following entities:

- Flight
- Passenger
- Airport

The attributes of each entity are also presented in the figure. The

Flight

Flight#	Destination	Origin	Distance	DOD	TOD
PK-124	Lahore	London	500	18-Sep-15	1700hrs

Passenger

Pid	Pname	Ticket#	Seat#	SeatType	Cost_ticket
1	Aamir	982	9B	Window	7000
2	Kashif	932	8A	Isle	6000

Airport

AID	Aname	Gate#	Runway#
1	Heathrow	12-B	7-A
2	Dubai	2-C	3-B

VU Gateway

normalization of the data will occur as follows:

DDDDDD. Non-repeating group:

i. Flight

EEEEEE. Repeating Groups:

i. Passenger
ii. Airport

First Normal Form (1NF)	
Table Name	**Attributes**
Flight	Flight#, destination, origin, distance, dod, tod
Passenger	Flight#,pid, pname, seat#, ticket#, cost
Airport	aid, Flight#,, gate#, aname, runway#

Second Normal Form (2NF)		
Entities	**Attributes**	**Comments**
Flight	Flight#, destination, origin, distance, dod, tod	No Change because there is single attribute Primary key.
Passenger	Flight#,pid, pname, seat#, ticket#, cost	No Change because each non-key is dependent full-functionally on Flight# and PID
Airport	aid, Flight#,, gate#, aname, runway#	Aid⇒gate#, aname, runway#, they have partial dependency on CPK (aid, flight#). As per rule following tables will be created in the place of airport table
Airport-Info	aid,gate#,aname,runway#	New Table
Airport	aid, fid	New Table

Third Normal Form (3NF)		
Entities	**Attributes**	**Comments**
Flight	Flight#, destination, origin, distance, dod, tod	Origin, Destination->distance, this is transitive dependency because to find out the correct value of distance values of origin, destination should be known and all three attributes are non-key.
Org-Dest	origin, destination,distance	New Table
Flight	Flight#,, origin, destination, dod,tod	New Table
Passenger	Flight#,pid, pname, seat#, ticket#, cost	No Change
Airport-Info	aid,gate#,aname,runway#	No Change
Airport	aid, fid	No Change

After applying the normalization up to third normal form (3NF) the data would be in the following shape:

VU-Gateway

TableName	Attributes
Flight	Flight#, destination, origin, distance, dod, tod
Org-Dest	origin, destination,distance
Flight	Flight#,, origin, destination, dod,tod
Passenger	Flight#,pid, pname, seat#, ticket#, cost
Airport-Info	aid,gate#,aname,runway#

VU-Gateway

Module 14: Denormalization

A. What is Denormalization?

Denormalization is the process of systematically adding redundancy in a database with the purpose of improving the database performance. The tables are merged into each other in order to group data as the more tables you have, the more joins you have to perform in your queries, and joins have a negative impact on performance and more joins mean more time is required to retrieve data because data is required from multiple table. The purpose of denormalization is to reduce the running time of, the time it takes the database management system (DBMS) to calculate the results.

FFFFFF. Need for Denormalization

Denormalization comes in as a solution if the running time of a database is of more importance than the database structure. In a large database, there are cases which require data from multiple tables in a single query and the application may need to perform this type of query hundreds of time in a minute. In such situation, a fully normalized database may be unacceptably slow. In such denormalization of database increase the performance of the database. So denormalization is a tradeoff between database structure and running time.

GGGGGG. [illegible]

The difference between normalization and denormalization is in the focus of these two activities. Normalization focus is to decrease the data redundancy in a database while on the other side the purpose or the focus of the denormalization is to decrease the running time of a database. As normalization reduce inconsistency, the database designer needs to ensure that the denormalized database does not become inconsistent.

HHHHHH. When to Denormalize?

Denormalization does not have any fix rule like there are rules in normalization. It is totally scenario or need based activity. As mentioned above, when the requirement is to reduce the running time of a database system, denormalization is done. It should be performed only if performance issues indicate that it is needed.

II. Scenarios for Denormalization

a. Storing End Date

The most common denormalization decision is to store the end date for periods that are consecutive; then the end date for a period can be derived from the start date of the previous period. It is appropriate when queries are needed from tables with long lists or records that are historical and user is interested in the most current record. Add an end date column to speed up queries so that they can use a between operator which is one of the efficient operator. Between is one of the most efficient operator that can be used to retrieve the data and it require start and end range.

b. Storing Details in Master Table

In a situation where the number of detail record in child table per master record are fixed (or has a fixed maximum) and where usually all detail records are queried with the master, you may consider adding the detail columns to the master table. This denormalization works best when the numbers of records in the detail table are small. This way you will reduce the number of joins during queries. The child table will be removed and it will be merged with the parent table. As this denormalization will occur, an anomaly will be created.

c. Example of Storing Details in Master Table

The phenomenon of storing details in the master table can be illustrated with the help of the following figure:

Before Denormalization

Order

oid	odate	amount
1	14-Sep-15	5000
2	18-Sep-15	9000

Order_detail

oid	pid	price
1	1	4000
1	2	1000

After Denormalization

oid	odate	amount	pid	price
1	14-Sep-15	5000	1	4000
1	14-Sep-15	5000	2	1000
2	18-Sep-15	9000		

Considering the above image, the normalization of the database resulted in two separate tables for Order and Order Details as shown in the figure. In order to speed up the query processing time, these two tables will be merged together i.e. denormalization. In this way no joins will be required, saving the space and speeding up the query processing time and ultimately improving the database performance.

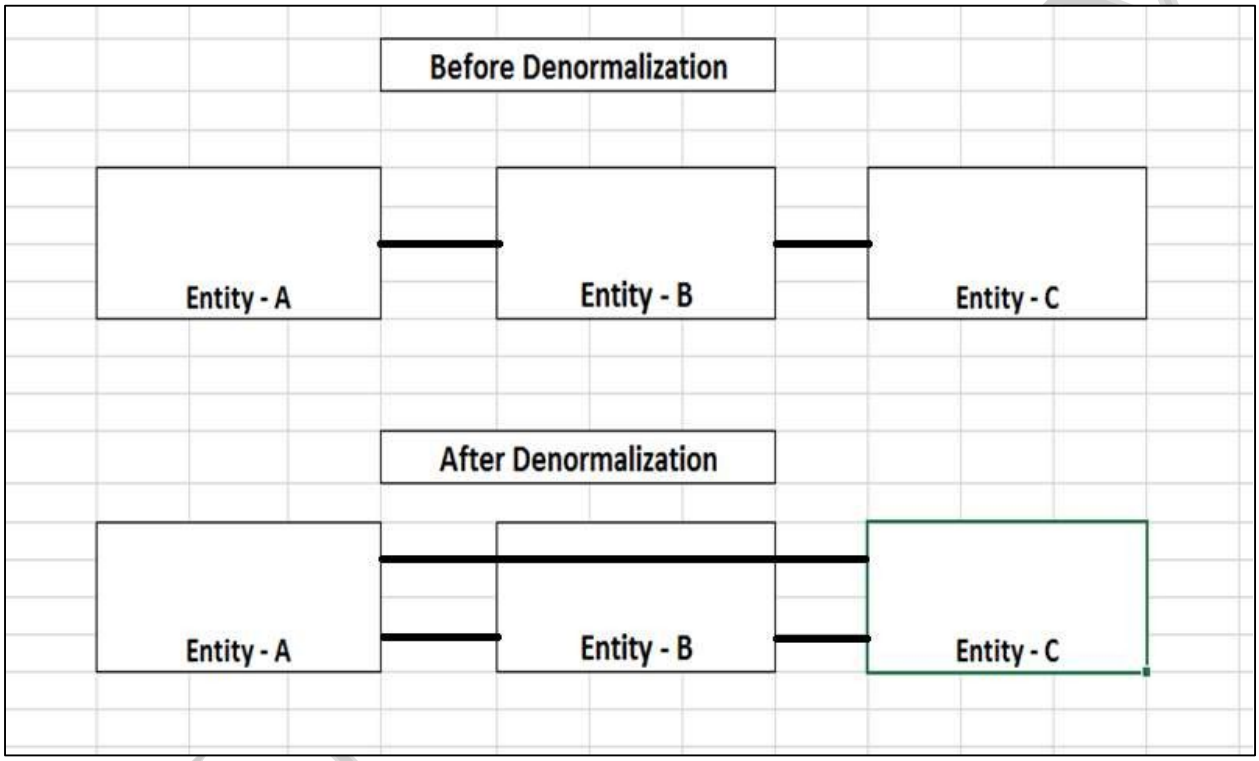
d. Short Circuit Key

The short circuit key ensures the relationships between the required entities. In a database that contains multiple levels of the master table and there is a need to query the lowest and highest level records only, consider creating short-circuit keys. These new foreign key definitions directly link the lowest level detail records to higher level grandparent records. The result can

produce fewer table joins when queries execute. It is appropriate when queries frequently require values from a grandparent and grandchild, but not from the parent.

e. Example of Short Circuit Key

As per the following example, before denormalization of the database, all the entities have parent-child relationship but no grandparent-grandchild relationship exist. In certain scenario where grandparent-grandchild is required to process the query, short circuit key comes in as a solution. As shown in the figure below, after denormalization, grandparent-grandchild also exist in the database. To achieve this, extra foreign keys will be required.



f. Current Indicator Column

This type of denormalization can be used in similar situations to the end date column technique. It can even be used in addition to an end date. It is a very common type of denormalization. Suppose most of the queries are to find the most current detail record. With this type of requirement, you could consider adding a new column to the details table to represent the currently active record. In simple words frequently updated column should have last-updated timestamp and a status column in added to show updated or old record.

g. Example of Current Indicator Column

Consider a table where the stock price of the company is being recorded. The stock price changes very frequently and the database also records the last time of update. But as the records changes frequently, so getting the latest stock price is difficult without having a status column in the table. This is where current indicator column is added (as a result of denormalization) with the motive of having status of all records. This column will tell whether a specific record / stock price are latest one or not. This whole scenario is illustrated in the figure below:

Before Denormalization

Cid	Cname	Stockprice	Last-updated
1	ABC	98	9:15
1	ABC	97.7	9:19
1	ABC	98.2	9:13

After DeNormalization

Cid	Cname	Stockprice	Last-updated	Status
1	ABC	98	9:15	Old
2	ABC	97.7	9:19	Latest
1	ABC	98.2	9:13	Old

h. Storing Calculated Value

When a calculation is frequently executed during queries, it can be worthwhile storing the results of the calculation. Queries that required calculation require more time to execute and to reduce this running time calculated fields should be pre-stored in the table. Through this type of denormalization the calculation does not need to be performed during a query and the source values do not need to be looked up every time the calculated value is required. As a result, query processing time will be shortened.

i. Example of Storing Calculated Value

Consider a database keeping record of cars with engine number, fuel tank size and average fuel consumption per liter. A query requiring total kilometers that can be traveled will require calculation which will ultimately increase the processing time. After denormalization, a column is added calculating the number of kilometers that a car can travel as per its fuel capacity. In this way, the calculation is pre stored and the calculation will not be required during query processing, reducing running time. This example is illustrated in the figure below:

Before DeNomralization

CarName	Engine#	TankSize in Liter	Average per Liter
City	93848293	40	17
Civic	9483823	45	15

After DeNormalization

				Calculated Field
CarName	Engine#	TankSize in Liter	Average per Liter	Total KM Travel
City	93848293	40	17	680
Civic	9483823	45	15	675

VU-Ga

Module 15: Introduction to Oracle 11g on Cloud

B. Introduction

It's a cloud service hosted by Oracle with full access to the features and operations that are available with Oracle Database, but Oracle hosts the VM and cloud storage. You can perform all database management and development operations—without purchasing and maintaining hardware, without knowing backup and recovery commands, and without having to perform such complex tasks as database software upgrades and patching.

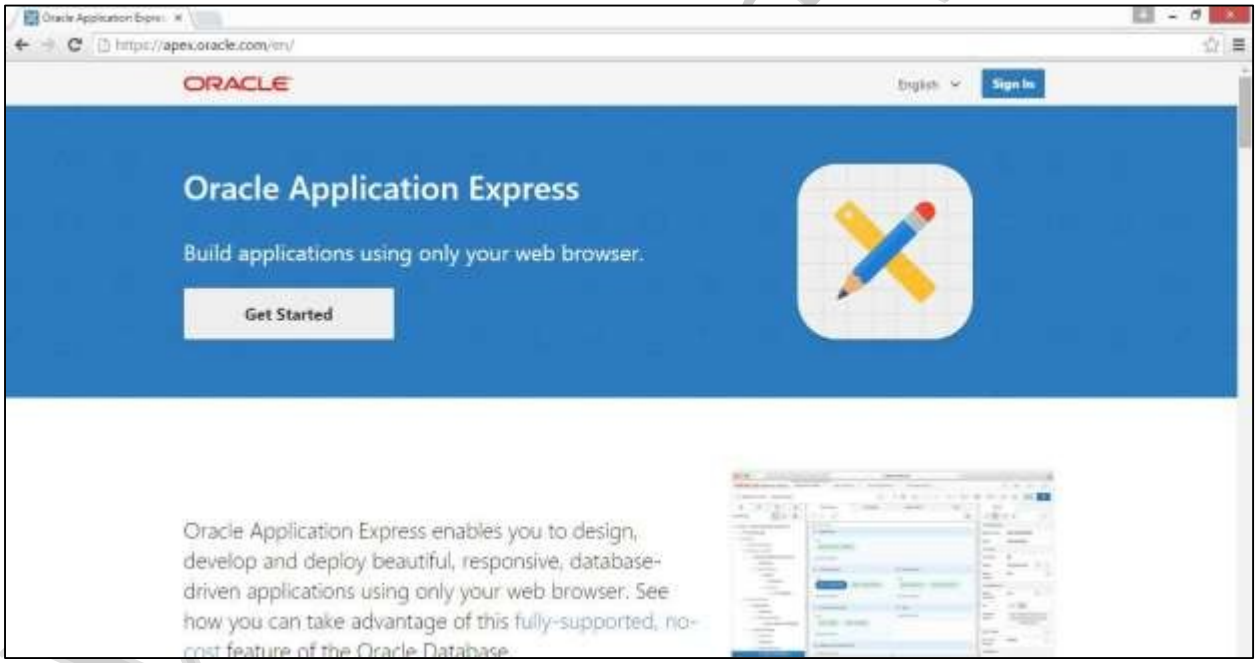
C. Login to Cloud

Login to the cloud will require the following steps:

Step 01:

Follow the following address to access Oracle 11g:

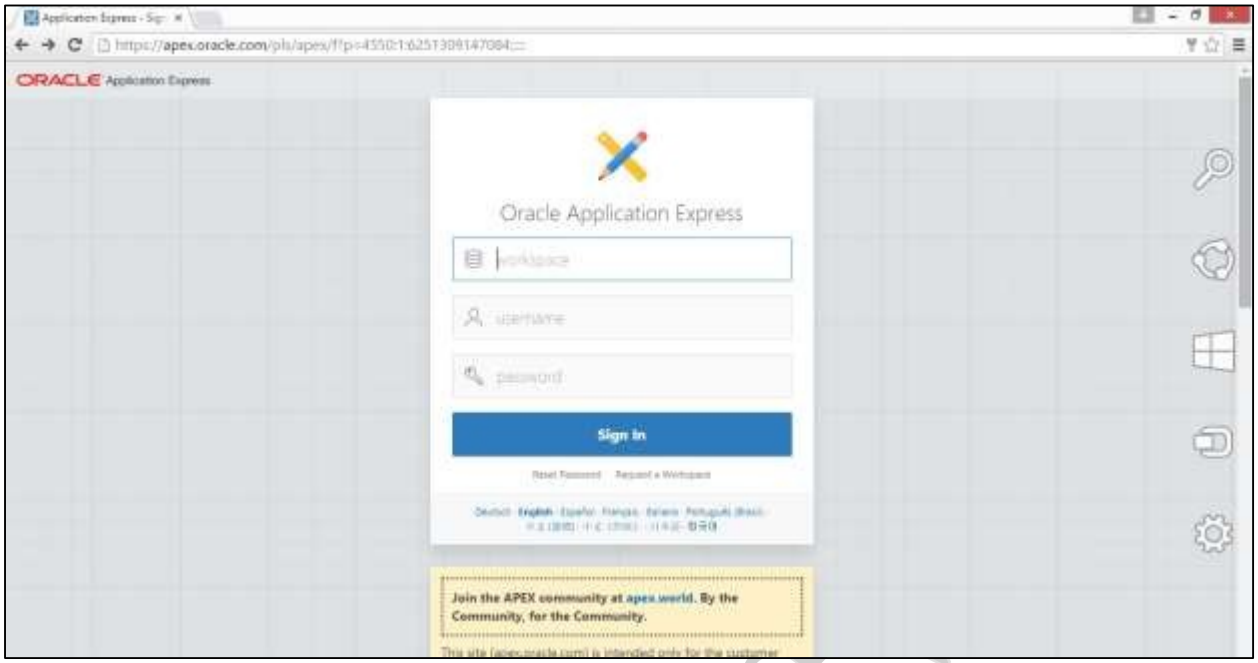
https://apex.oracle.com/en/



Step 02:

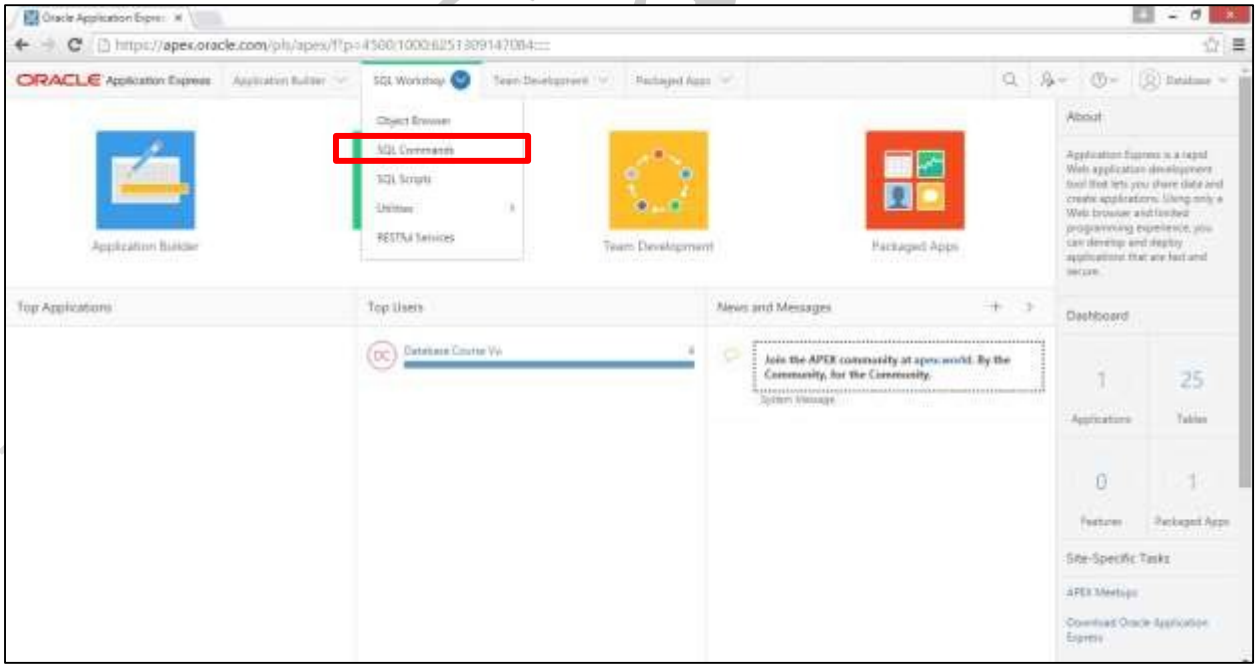
Click on the Sign In button on the upper right corner of the page and enter the following credentials:

Workspace Name	
Username	
Password	



Step 03:

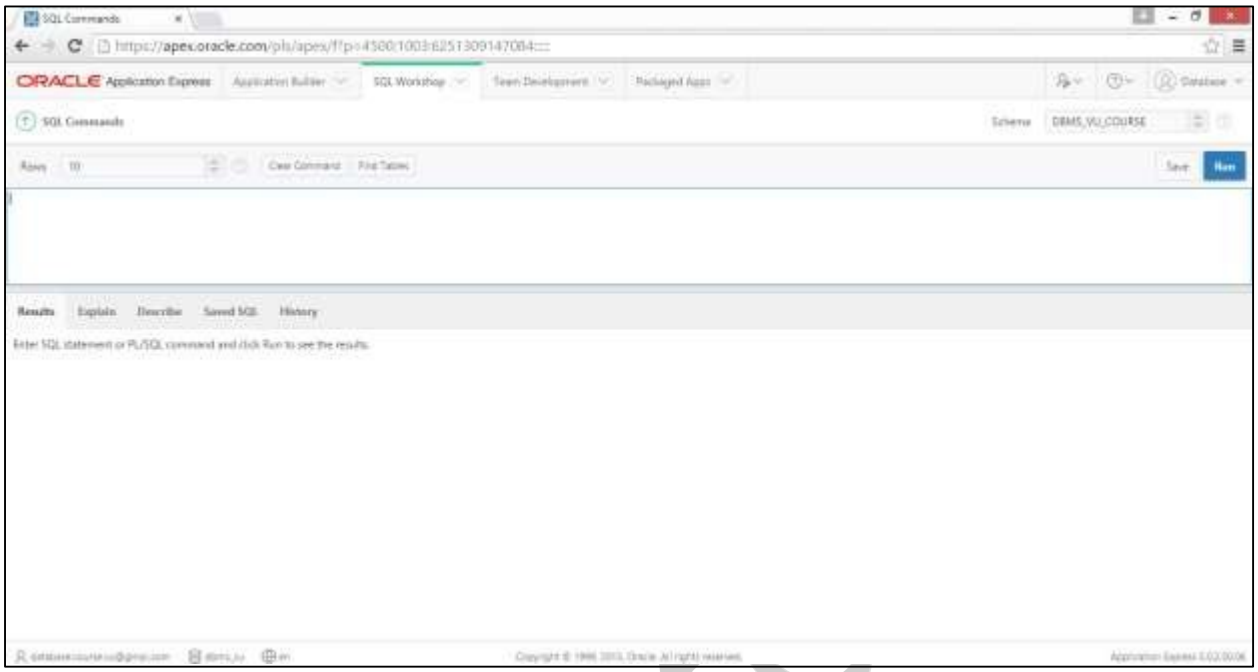
- Click on the SQL Workshop tab at the top-mid of the page



Step 04:

- Click on SQL Commands tab from the drop down list to access the code or enter SQL Statements / Commands and click Run to see the results

- Click on History Tab to access the saved code



VU-Gate

Module 16: Using Data Definition Language (DDL) in Oracle 11g

Data Definition Language (DDL) is used to create database object in the Database. Database Object has Data dictionary created and managed by System in the Database. Table is the most important database important which is required to storage and retrieve data. DDL include Create, Alter and Drop statements.

A. Syntax of DDL

General syntax of Create Table statement is as follow:

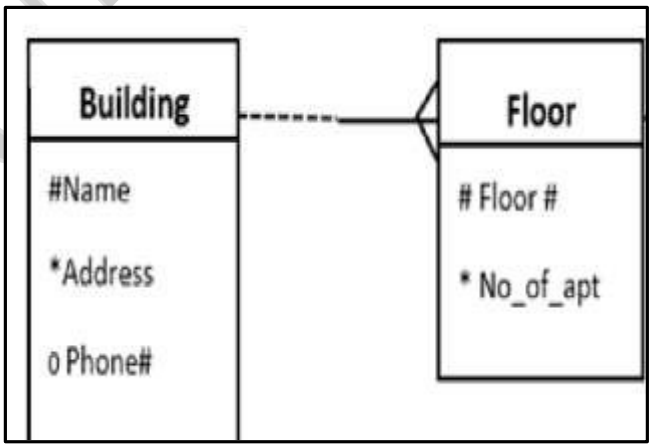
Create Table TableName (Column datatype [null | not null], ….)

DataType:

i. Number
ii. VarChar2 - Variable Length Character
iii. Date – DD/Mon/YYYY

B. Creating Single Table

To implement the single table (building table) from the ERD given below:



VU-Gateway

DDL statement to implement Building Table:

- Create table Building (bname varchar2 (30) primary key, address varchar2 (40), phone number number (10));

C. Creating Table with Foreign Key

In floor table, bname is foreign key, Floor# and bname is Composite Primary key. Code to implement the table is as below:

- Create table Floor (floorno number (10), no_of_apt number (10), bname varchar2 (30) references building (bname), primary key (floorno, bname));

D. Alter Table Command

Alter command is used to either add new column or modify data type or size in existing table

Adding New Column in Building Table

- Alter table Building add (rent number (10));

To modify data type of phone# from number to varchar2, following code can be used:

Alter table Building modify (phoneno varchar2 (10));

E. Drop Table Command

To drop table from the database, drop command can be used. While dropping the table it is important to maintain the sequence to drop the table. All the child table should be dropped first and then parent table can be drop.

- Drop table floor;
- Drop table building;

VU-Gateway

Module 17: Using Data Manipulation Language (DML) and Data Control Language (DCL)

A. Insert Statement Syntax

The insert statement is used to add data in one or more tables in a database. The basic syntax of the insert statement is as follows:

```
INSERT into TableName
VALUES (value 1, value 2…)
```

Every column in list-of-columns must have a valid value in the corresponding position in list-of-values. Therefore, before you insert a row into a table, you must know what columns the table has, and what their valid values are. Referential Integrity constraints are implemented by Default.

JJJJJJ. Implementing Insert Statement

Consider the following relation model to illustrate the example:

Entity	Attirbutes
Building	Building Name***(bname)***, Address, Phone Number
Floor	Floor Number, Number of Apartments, Bname

Continuing with the above example, the example of insert statement would be:

```
INSERT into building
VALUES ('Test', 'VU', 3948283);
```

KKKKKK. Viewing Data from Table

To verify the insertion of data in the table following query can be used

Select * from building;

Note: Detail on how to use Select will be discussed in detail later in the course

LLLLLL. Limited Column Data Insertion

If value of limited columns of the table are known then in this case limited values can be added in the table, but value of Primary Key column can't be left NULL.

```
INSERT into building (bname, address)
VALUES ('Test-1', 'VU -LRO');
```

MMMMMM. Update Table Syntax

Update command is used to update one or more column of a table. Following is its general syntax:

```
UPDATE TableName
SET colum1 = value, column2 = value
WHERE = condition
```

Each value must be valid for its column-name. If you include the WHERE clause, the statement updates column values only in rows that satisfy condition.

NNNNNN. Implementing Single Column Update Statement

```
UPDATEemp
SET ename='Anders'
WHERE empno=7369;
```

This will update the Employee Name to Anders of employee no: 7369

To verify the changes following command can be used:

```
SELECT * from emp;
```

OOOOOO. Implementing Multiple Column Update Statement

```
UPDATE emp
SET job ='MANAGER' ,sal = 2000
WHERE empno=7369;
```

This will update Job and Sal column of the Emp table

To verify the changes following command can be used:

```
SELECT * from emp;
```

PPPPPP. Delete Statement Syntax

The Delete statement is used to delete one or more row from a table. The deleted rows can be rollback if not committed. Being committed means that all the changes in the database has been made permanent using Commit Command. Rollback command is used to restore the changes

made in the database to the way it was before, (this only works if you haven't already used Commit command). Delete command can also be used with condition to delete a particular row.

```
DELETEFROM Table Name;
DELETEFROM Table Name WHERE condition 1;
```

QQQQQQ. Implementing Delete Statement

```
DELETEFROM empWHERE empno=7369;
```

The command below will delete all the rows from table emp:

```
DELETE FROM emp;
```

RRRRRR. Truncate Table

TRUNCATE TABLE removes all rows from a table, but the table structure and its columns, constraints, indexes, and so on remain. Removing rows with the TRUNCATE TABLE statement can be faster than removing all rows with the DELETE statement, especially if the table has numerous triggers and other dependencies.

SSSSSS. Transaction Basics

A transaction is a unit of work that is performed against a database. Transactions are units or sequences of work accomplished in a logical order, whether in a manual fashion by a user or automatically by some sort of a database program.

A transaction is the propagation of one or more changes to the database. For example, if you are creating a record or updating a record or deleting a record from the table, then you are performing transaction on the table. It is important to control transactions to ensure data integrity and to handle database errors.

TTTTTT. Commit Statement

Commit Statement is used to save the changes made in the database. The Commit Statement is the transactional command used to save changes invoked by a transaction to the database. The Commit Statement saves all transactions to the database since the last Commit or Rollback Statement. The DML statements are not auto commit and should be saved by giving the commit statement.

UUUUUU. Rollback Statement

The Rollback Statement is the transactional command used to undo transactions that have not already been saved to the database. The Rollback Statement can only be used to undo transactions since the last Commit Statement or Rollback Statement was issue d. In very simple words, Used to undo the work performed by the current transaction.

Module 18: Structured Query Language (SQL) Basics

A. Structured Query Language (SQL)

SQL is a database computer language designed for the retrieval and management of data in relational database. SQL stands for Structured Query Language.SQL consists of a data definition language (DDL), data manipulation language (DML), data retrieval language (DRL) and a data control language (DCL). The scope of SQL includes data insert, query, update and delete, schema creation and modification, and data access control.

XXX. SQL Syntax

All the SQL statements start with any of the keywords like SELECT, INSERT, UPDATE, DELETE, ALTER, DROP, CREATE, USE, SHOW and all the statements end with a semicolon (;).

```
SELECT DISTINCT * ColumnName FROM TableName;
```

- * mean all the columns
- ColumnName is one or more column from table
- Distinct mean unique values from column

WWWWWW. Implementing SQL

Few sample statements of SQL are as follows where the words in capital letters are SQL commands or statements and words in capital letters are table names.

The following command will display all the values from Deptno column of Emp table

```
SELECT depno FROM emp;
```

The following command will display all the unique values from Deptno column of Emp table

```
SELECT DISTINCT deptno FROM emp;
```

The following command will display values from all column with all the row mean it will display the whole table

```
SELECT * FROM emp;
```

The following command will display ename and job column from emp table

```
SELECT ename, job FROM emp;
```

XXXXXX. SQL and WHERE Clause

The WHERE clause includes a condition, which restricts the rows returned by the query. The WHERE clause eliminates all rows from the result set where the condition does not evaluate to True. From a large table, required rows can be fetch by using WHERE clause which applied to each row of the table. If the given condition is satisfied then only it returns specific value from the table. You would use WHERE clause to filter the records and fetching only necessary records.

YYYYYY. Implementing SQL and WHERE Clause

Example 01: Consider a situation where you need to get list of all employees (from the employee table) who are titled as managers in the job column. The code will be as follows:

```
SELECT ename, job FROM emp
WHERE job ='MANAGER';
```

Example 02: In a situation where you need to get list of all employees (from the employee table) whose salary is not equal to 2000. The code will be as follows:

```
SELECT ename, sal FROM emp
WHERE sal<> 2000;
```

ZZZZZZ. SQL and Logical Operators

Logical operators are used to specify multiple conditions in a single WHERE clause to, say, retrieve rows based on the values in multiple columns. For this purpose AND, NOT and OR operators are used to combine two or more conditions into a compound condition to get the specific required results. These operators combine two conditions at a time to select a value for output. The details of the logical are as follows:

Logical Operators	Description	Syntax
AND	Returns TRUE if both conditions are TRUE.	SELECT * FROM EMP WHERE job='CLERK' AND deptno=10
NOT	Reverses the value of the condition	SELECT * FROM EMP WHERE NOT (job IS NULL)
OR	Returns TRUE if either condition is TRUE.	SELECT * FROM emp WHERE job='CLERK' OR deptno=10

The following table shows the result of comparing two conditions with Logical Operator:

Truth Table				
Condition 01	**Condition 02**	**OR**	**AND**	**Description**
TRUE	TRUE	TRUE	TRUE	If both conditions are TRUE, AND/OR will results in TRUE
TRUE	FALSE	TRUE	FALSE	If one condition is TRUE and other is FALSE, OR Operator will result TRUE, AND Operator will results FALSE
FALSE	TRUE	TRUE	FALSE	
FALSE	FALSE	FALSE	FALSE	If both conditions are FALSE, AND/OR will results in FALSE

AAAAAAA. Implementing Logical Operator 01

Consider the following scenario:

Scenario: Write a query to find out list of all employee names who are earning more than 2500 but less than 5000.

```
SELECT ename, sal FROM emp
WHERE sal> 2500 AND sal< 5000;
```

BBBBBBB. Implementing Logical Operator 02

Scenario: Write a query to find out all those employees who are working in Dept # 20 with designation of Analyst

```
SELECT * FROM emp
WHERE deptno=10 AND job='ANALYST';
```

CCCCCCC. Implementing Logical Operator 02

Consider the following scenarios:

Scenario 01: Write a query to display list of all those empno who are earning more than 400 as commission with designation of Manager

```
SELECT empno FROM emp
WHERE comm> 400 and JOB='MANAGER';
```

Scenario 02: Write a query to find out all sales man working in dept # 30 but their salary is less than 1500

```
SELECT * from emp;
SELECT * from emp WHERE JOB='SALESMAN' AND deptno=30 and sal< 1500;
```

DDDDDDD. Wildcard Characteristics in SQL

To broaden the selections of a SQL statement, two wildcard characters, the percent sign (%) and the underscore (_), can be used. The percent sign is analogous to the asterisk (*) wildcard

character used with MS-DOS. The percent sign allows for the substitution of one or more characters in a field. The underscore is similar to the MS-DOS wildcard question mark character. The underscore allows for the substitution of a single character in an expression. Wild cards are used where part of value to be searched is known not the exact value mean if pattern is known not the complete value. Wild cards are used to make regular expressions to identify patterns in the queries. There are two wild cards as below:

%: Zero or more characters

_: Exactly one character

EEEEEEE. Like Operator in SQL

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column. Wildcard Operators (% and _) are used to identify the patterns. The SQL LIKE condition allows you to use wildcards to perform pattern matching. The LIKE condition is used in the WHERE clause of a SELECT, INSERT, UPDATE, or DELETE statement.

FFFFFFF. LIKE Syntax

The basic syntax of LIKE clause is as follows:

```
SELECT column_name(s)
FROM table_name
WHERE column_name LIKE pattern and column_name LIKE pattern;
```

The basic Syntax of LIKE with Percentage (%):

```
SELECT FROM table_name
WHERE column LIKE 'XXXX%'
```

The basic Syntax of LIKE with Underscore (_):

```
SELECT FROM table_name
WHERE column LIKE 'XXXX_'
```

GGGGGGG. Implementing Like – 01

Consider the following Scenario:

Scenario: Write a query to display list of name of all those employees who are having either E in the name or the name should end with G with at least two characters

```
SELECT * FROM emp
WHERE ename LIKE‘%E%’ OR ename LIKE‘%-G’;
```

HHHHHHH. Implementing Like – 02

Consider the following Scenario:

Scenario: Write a query to display all information about all those employees who are having RE in the job with at least three characters in job and should be earning at least 2500 but at most 5000.

```
Select * FROMemp
WHERE job LIKE‘%RE-%’ AND sal> 2500 AND sal< 5000;
```

Practice Scenario:

- Write a query to display all information about all those employees who are at least one vowel in the ename?
- Write a query to display list of those employees who are earning more than 2000 but at most salary is 5998 and are having at least two occurrence of E in the ename?

III. IN Operator

The IN operator is SQL, works like OR Operator, allows to Search for a value from given list of value. The basic syntax of IN Operator is shown below:

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1,value2,...);
```

JJJJJJJ. Implementing IN Operator

Consider the following scenario:

Scenario: Write a query to display all information about all those employees working either as MANAGER or ANALYST or CLERK and should belong to department number 20 and should be earning at most 2500

```
SELECT * FROM emp
WHERE job IN (‘MANAGER’, ‘ANALYST’, ‘CLERK’) AND deptno = 20 AND
sal<=2500;
```

KKKKKKK. BETWEEN Operator

The BETWEEN operator selects values within a range, the values can be numbers, text, or dates. The border values of the range are included in the search. The between operator takes two values: minimum and maximum values and search between the two including the min and max values. The basic syntax of BETWEEN operator is as follows:

The general syntax of the between logical operator is:

```
Select * from table_name
Where col_name BETWEEN val1 AND val2
```

In above syntax val1 and val2 can be two numbers, characters or dates.

LLLLLLL. Implementing BETWEEN Operator

Here is the example of BETWEEN Operator:

Scenario: Write a query to display all information about all those employees who are earning at least 2975 and at most 5000 and are working in department number 20 and are reporting to employee with empno 7839

```
SELECT * FROM emp
WHERE sal BETWEEN 2975 AND 5000 AND deptno= 20 AND mgr = 7839
```

MMMMMMM. Single Row Function

The Single Row Function operates on single rows only and returns one result per row. Single-row functions are used to manipulate data items. They accept one or more arguments and return one value for each row returned by the query. Single row functions can be character functions, numeric functions, date functions, and conversion functions. These functions require one or more input arguments and operate on each row, thereby returning one output value for each row. Single row functions can be used in SELECT and WHERE statement.

NNNNNNN. Implementing Single Row Function

ROUND and TRUNC:

```
SELECTROUND(194.683,1), TRUNC(194.683,1) FROM dual;
```

LENGTH, INSTR & CONCAT:

```
SELECTLENGTH (ename), INSTR (ename, 'E'), CONCAT(ename,job) FROM emp;
```

OOOOOOO. Group or Multiple Row Function

These functions manipulate groups of rows to give one result per group of rows. Group functions compute an aggregate value based on a group of rows. The example of the group or multiple row function is given below:

```
SELECT COUNT *, sum (sal), min(sal), max (sal),Avg (sal) FROM emp;
```

PPPPPPP. GROUP BY Clause

GROUP BY clause group together similar row together to form group and the multiple row function is used with GROUP BY Clause. The SQL GROUP BY clause is used in collaboration with the SELECT statement to arrange identical data into groups. The basic syntax of GROUP BY clause is given below.

```
SELECT column1, column2
FROM table_name
WHERE [ conditions ]
GROUP BY column1, column2
```

QQQQQQQ. Implementing GROUP BY Clause 01

Situation: What is the salary paid of each department?

Steps to Solution:

- Need to group together all rows of each department separately i-e creating groups
- Need sum of salary each department – group

```
SELECTSUM (sal), deptno FROM emp
GROUP BY deptno;
```

The complete steps are shown below:

Step-1

Deptno	Sal	Group #
10	5000	Group -1
20	1200	Group -2
30	5002	Group-3
10	2400	Group -1
10	1900	Group-1
20	2100	Group-2

Step-2

Group #	No of Rows	Group Function: Sum of Sal - Output is required by Query
1	3	5000+2400+1900 = 9300
2	2	1200+2100 = 3300
3	1	5002

The final output of the query is shown in the figure below:

Output of Query

Deptno	Sum of Salary
10	9300
20	3300
30	5002

RRRRRRR. Implementing GROUP BY Clause 02

Consider the following:

Scenario: What is average and maximum salary paid to each Job who are reporting to MGR 7839?

Solution – 1

```
SELECT avg (sal), maximum (sal) FROM emp
WHERE mgr = 7839
GROUP BY job;
```

Solution – 2

```
SELECT avg (sal), max(sal), job, mgr, sal FROM emp
WHERE mgr=7839
GROUP BY job
```

Result – Error job, mrgare not written after group by clause

SSSSSSS. HAVING Clause

The having clause, is just like the where clause, that filters the results in aggregated / grouped data. The Where clause cannot be used in the aggregated data, so SQL having clause is introduced to filter the results. The HAVING clause enables you to specify conditions that filter which group results appear in the final results.

The WHERE clause places conditions on the selected columns, whereas the HAVING clause places conditions on groups created by the GROUP BY clause.

The basic syntax is as follows:

```
SELECT column1, column2
FROM table1, table2
GROUP BY column1, column2
HAVING [ conditions ]
```

TTTTTTT. Implementing HAVING Clause – 1

Scenario: Write a query to display average salary of each department if there are at least 2 employees working in the department

Steps to Solution:

- Need to group together all rows of each department separately i-e creating groups
- Need sum of average each department – group
- Count number of employees in each group
- Display those groups in which there are more than 2 rows

Solution:

```
SELECT avg (sal) FROM emp
GROUP BY deptno
HAVING count (*) > 3;
```

UUUUUUU. Implementing HAVING Clause – 2

Scenario: Write a query to display maximum and minimum salary by each department if average salary is more than 1500 of the department and less than 3000. The employee should not be included if there is any occurrence of 'A' in the ename.

Solution:

```
SELECT max (sal), min (sal) FROM emp
WHERE ename NOT LIKE'%A%'
GROUP BY deptno
HAVING avg (sal) > 15000 AND avg (sal) < 3000
```

VVVVVVV. ORDER BY Clause

The Order by clause is used with the SQL SELECT statement to sort the results in ascending or descending order. You have to specify one or more columns for what you want to sort the table result set. The ORDER BY keyword sorts the records in ascending order by default. To sort the records in a descending order, you can use the DESC keyword. The basic syntax is as follows:

```
SELECT column_name,
FROM table_name
ORDER BY column_name ASC|DESC,
```

WWWWWWW. Date Handling

Date is format specific in SQL and use a specific format i.e. MM-DD-YY. Two dates can't be added, subtracted, multiply and divide.Days can be added to Date only.

```
SELECT hiredate + 30, hiredate-50, sysdate, comm, sal, deptno FROM emp WHERE
deptno=30 AND comm is NOT null OR sal< 3000;
```

XXXXXXX. Date Functions

Following are the date functions in SQL with their descriptions and syntax:

Function	Description & Syntax
MONTHS_BETWEEN	The MONTHS_BETWEEN function returns the number of months between *date1* and *date2* Syntax: MONTHS_BETWEEN(date1, date2)
ADD_MONTHS	The ADD_MONTHS function returns a date with a specified number of months added. Syntax: ADD_MONTHS(date1, number_months)
NEXT_DAY	The NEXT_DAY function returns the first weekday that is greater than a *date*. Syntax: NEXT_DAY(date, weekday)
LAST_DAY	The LAST_DAY function returns the last day of the month based on a *date* value. Syntax: LAST_DAY(date)

An example of date function can be assumed to be as follows:

```
SELECT MONTHS_BETWEEN (sysdate, hiredate)/12, ADD_MONTHS (sysdate,
hiredate), NEXT_DAY (sysdate, 'Wednesday), LAST_DAY (sysdate) FROM emp;
```

VU-Gateway

Module 19: Advance SQL

A. Cartesian Product

Cartesian product is mathematically a binary operation in which two objects or sets (or tables) are combined in an "everything in combination with everything" fashion. In SQL statement, a Cartesian product is where every row of the first table is joined with every row of the second table. Simply defining the Cartesian product, pairing of one element in set with every element of second set I called Cartesian product. The whole concept is further explained by the following example:

Example:

Suppose the following two sets:

- Set A = {2, 5, 6}
- Set B = {8, 1}

Cartesian product of the above two sets would be:

- A * B = {(2,8), (2,1), (5,8), (5,1), (6,8), (6,1)
- Total Number of Pairs = No. of Element in A * No. of Element in B = 3 * 2 = 6

YYYYYYY. Cartesian Product & Joins

As mentioned above, Cartesian product is the pairing of one element in one set with every element of other set. But, as the relational databases are usually normalized, combining two tables creates another table which combines the information from the both tables. This will result in duplication in the data, losing the data integrity. Joining tables will show wrong data. Go through the following example to understand the whole concept.

Example: Employee Database

Consider the following table in an employee database:

Empno	Ename
1	Imran
2	Kashif
3	Asif

Deptno	Dname
10	Sales
20	Marketing
30	Engineering

The output of a Cartesian product would be:

Output of Cartesian Product: **Total Rows = 3 * 3 = 9**

In an SQL Statement:

```
SELECT empno,ename, d.deptno, dname
FROMemp e, dept;
```

The output of the product will be the total number of rows in employee table multiplied with the total number of rows in department table means every employee is working in every department which is **WRONG** information. In an equation format:

No. of Rows in EMP * No. of Rows in DEPT = 3*3 = 9

ZZZZZZZ. Join or Inner Join

Joins are required when data from multiple tables is required. The standard join operation is knows as inner join. It horizontally combines two or more tables into a single working table. An inner join is performed by logically performing the Cartesian product (generating all combinations of rows) of the tables.

A primary key field in one table can be foreign key field in another table and a join operation is used to combine tables using a common key in both tables.

Basic Join Statement (Continuing with the example of Employee Database):

```
SELECT empno,ename, d.deptno, dname
FROMemp e, dept d
WHEREd.deptno=e.deptno;
```

Joins with Other Conditions:

```
SELECT empno,ename, d.deptno, dname,
ROUND (months_between(sysdate, hiredate),0), hiredate
FROM emp e, dept d
WHEREd.deptno=e.deptnoANDmonths_between(sysdate, hiredate) > 410 ;
```

AAAAAAAA. Self Join

A self-join is a query in which a table is joined (compared) to itself. Self-joins are used to compare values in a column with other values in the same column in the same table. In self-join a table is joined with itself, especially when the table has a FOREIGN KEY which references its own PRIMARY KEY. To join a table itself, means that each row of the table is combined with itself and with every other row of the table. Self joins are used in a recursive relationship. To explain that further, think of a COURSE table with columns including PREREQUISTE, COURS_NO and others. There is a recursive relationship between PREREQUSITE and COURSE_NO as PREREQUSITE is valid only if it is also a valid COURSE_NO. the basic syntax of self-join is:

```
SELECT a.column_name, b.column_name...
FROM table1 a, table1 b
WHERE a.common_field = b.common_field;
```

BBBBBBBB. Implementing Self-Joins

Example 01:

```
SELECT e.ename,e.empno, b.ename, b.empno
FROM emp e, emp b
WHERE e.empno=b.mgr;
```

Example 02:

Consider a Customer Table with the following attributes:

- Table: Customer (id, name, age, address, salary)

The code for the Self-Join would be:

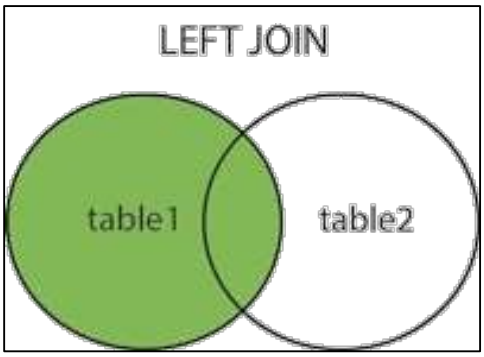
```
SELECT a.ID, b.NAME, a.SALARY
FROM CUSTOMERS a, CUSTOMERS b
WHERE a.SALARY<b.SALARY;
```

CCCCCCCC. Outer Join

An outer join extends the result of a simple join. An outer join does not require each record in the two joined tables to have a matching record. The joined table retains each record—even if no other matching record exists, non-matching as NULL. The SQL OUTER JOIN returns all rows from both the participating tables which satisfy the join condition along with rows which do not satisfy the join condition. The SQL OUTER JOIN operator (+) is used only on one side of the join condition. Outer joins subdivide further into left outer joins and right outer joins, depending on which table's rows are retained.

DDDDDDDD. Left Outer Join

A left outer join returns all the values from an inner join plus all values in the left table that do not match to the right table, including rows with NULL (empty) values in the link field. All the rows from the left table will be included and all the matching rows from the right table will be included. A NULL value will be displayed for a non-matching value. Simply defining, the result of a left outer join (or simply left join) for tables A and B always contains all records of the "left" table (A), even if the join-condition does not find any matching record in the "right" table (B).



EEEEEEEE. Implementing Left Outer Join

Suppose the following example to understand the concept:Suppose a table called suppliers with two fields: supplier_id and supplier_name) and another table called the order table with three fields (order_id, supplier_id and order_date). These two tables contain the following data:

supplier_id	supplier_name
10000	IBM
10001	Hewlett Packard
10002	Microsoft
10003	NVIDIA

Suppliers Table

order_id	supplier_id	order_date
500125	10000	2003/05/12
500126	10001	2003/05/13

Orders Table

If we run the following SQL statement, containing left outer join:

```
SELECT supplier.sid, supplier.sname, orders.doe
FROM supplier, orders
WHERE supplier.sid = orders.sid(+);
```

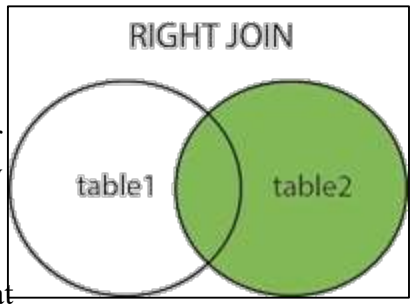
The result set will look like this:

supplier_id	supplier_name	order_date
10000	IBM	2003/05/12
10001	Hewlett Packard	2003/05/13
10002	Microsoft	<null>
10003	NVIDIA	<null>

The rows for Microsoft and NVIDIA will be included as full outer join was used but the outer_date filed contain the NULL value for these two.

FFFFFFFF. Right Outer Join

The RIGHT JOIN keyword returns all rows from the right table (table2), with the matching rows in the left table (table1). The result is NULL in the left side when there is no match. A right outer join (or right join) closely resembles a left outer join, except with the treatment of the tables reversed. Every row from the "right" table (B) will appear in the joined table at least once. If no matching row from the "left" table (A) exists, NULL will appear in columns from A for those records that have no match in B.



GGGGGGGG. Implementing Right Outer Join

Consider the following example: again we have two tables; Supplier and Orders. The tables contain the following data.

supplier_id	supplier_name
10000	Apple
10001	Google

Suppliers Table

order_id	supplier_id	order_date
500125	10000	2013/08/12
500126	10001	2013/08/13
500127	10002	2013/08/14

Orders Table

If we run the following SQL statement, containing full outer join:

```
SELECT supplier.sid, supplier.sname, orders.doe
FROM supplier, orders
WHERE supplier.sid(+) = orders.sid;
```

The result will be:

order_id	order_date	supplier_name
500125	2013/08/12	Apple
500126	2013/08/13	Google
500127	2013/08/14	<null>

The row for 500127 (order_id) would be included because a RIGHT OUTER JOIN was used. However, you will notice that the supplier_name field for that record contains a <null> value.

HHHHHHHH. Subquery Basics

A subquery is a query within a query. Subqueries enable you to write queries that select data rows for criteria that are actually developed while the query is executing at run time. SQL subquery is usually added in the WHERE Clause of the SQL statement. Most of the time, a subquery is used when you know how to search for a value using a SELECT statement, but do not know the exact value in the database. Subqueries are an alternate way of returning data from multiple tables, or simply, alternative of joins. The general syntax is as follows:

```
SELECT *
FROM t1
WHERE column1 = (SELECT column1 FROM t2);
```

IV. Implementing Subqueries

Consider the following scenarios:

Scenario 01: Write a query to display information of all those employees who are earning minimum salary.

Solution:

```
SELECT ename, sal, deptno
FROM emp
WHERE sal = (SELECT MIN (sal) FROM emp);
```

Scenario 02: Write a query to display all those deptno where minimum salary is less than average salary of all the salary among all the employee.

Solution:

```
SELECT deptno, MIN (sal)
FROM employees
GROUP BY deptno
HAVING MIN (sal) < (SELECT AVG (sal)
FROM employees)
```

VU-Gateway

Module 20: Database Views and Data Dictionaries in Oracle 11g

A. Motivation for Views

A view can be thought of as either a window of table or a virtual table. A view is a logical representation of another table or combination of tables. A view derives its data from the tables on which it is based; View does not have its own data. These tables are called base tables. Base tables might in turn be actual tables or might be views themselves. The SQL VIEW is, in essence, a virtual table that does not physically exist. Rather, it is created by a pre-defined SQL statement that joins one or more tables.

Access to data can be improved using SQL views as they Summarize data from various tables which can further be used to generate reports. Views are very powerful because they allow you to tailor the presentation of data to different types of users. While views can help to obscure large queries from users and to standardize data access, a view can also draw data from several different tables and present it as a single table, turning multi-table queries into single-table queries against the view and the queries can be reused to reduce time. All the DML statement can be issued against a view.

JJJJJJJJ. Syntax of a View

The following is a general syntax of the view:

```
CREATE VIEW view_name AS
SELECT columns
FROM tables
WHERE conditions;
```

KKKKKKKK. Writing Views

Suppose we have EMP and DEPT table. To see the empno, ename, sal, deptno, department name and location we have to give a join query like this.

```
CREATE VIEW emp_det AS
SELECT e.empno, e.ename, e.sal, e.deptno, d.dname, d.loc
FROM emp e, dept d
WHERE e.deptno=d.deptno;
```

This SQL CREATE VIEW example would create a virtual table based on the result set of the select statement. You can now query the SQL VIEW as follows:

```
SELECT * FROM emp_det;
```

This will show same result as you have type the long join query. Now you can treat this EMP_DET view same as any other table.

LLLLLLLL. Writing Complex Views

WITH CHECK OPTION creates the view with the constraint that INSERT and UPDATE statements issued against the view are not allowed to create or result in rows that the view cannot select. Consider the following example:

Suppose all the employee working in Department No. 10 belongs to accounts department and most of the time you deal with these people. So every time you have to give a DML or Select statement you have to give a WHERE condition like WHERE DEPTNO=10. To avoid this, you can create a view as given below

```
CREATE VIEW accounts_staff AS
SELECT Empno, Ename, Deptno
FROM Emp
WHERE Deptno = 10
WITH CHECK OPTION CONSTRAINT ica_Accounts_cnst;
```

Now to see the account people you don't have to give a query with where condition you can just type the following query.

```
SELECT * from accounts_staff;
```

The query that defines the ACCOUNTS_STAFF view references only rows in department 10.

Considering the example above, the following INSERT statement successfully inserts a row into the EMP table through the ACCOUNTS_STAFF view:

```
INSERT INTO Accounts_staff VALUES (110, 'ASHI', 10);
```

However, the following INSERT statement is rolled back and **returns an error** because it attempts to insert a row for department number 30, which could not be selected using the ACCOUNTS_STAFF view:

```
INSERT INTO Accounts_staff VALUES (111, 'SAMI', 30);
```

MMMMMMMM. Data Dictionary Concepts

Data dictionary is a read-only set of tables that provides information about its associated database. A data dictionary contains the definitions of all schema objects in the database (tables, views, indexes, clusters, synonyms, sequences, procedures, functions, packages, trigger, and so on). A dictionary, as said above, is built in Tables which are created and maintained by System.

The data dictionary has three primary uses:

- Oracle accesses the data dictionary to find information about users, schema objects, and storage structures.
- Oracle modifies the data dictionary every time that a data definition language (DDL) statement is issued.
- Any Oracle user can use the data dictionary as a read-only reference for information about the database.

Using Data Dictionary Concepts

Few data dictionary views are mentioned in the table below:

Data Dictionary View	
SELECT * FROMuser_tables;	ALL_TABLES describes all relational tables accessible to the user.
SELECT * FROMuser_tables;	ALL_CONSTRAINTS describes constraint definitions on tables accessible to the current user.
SELECT * FROMuser_constraints;	USER_CONSTRAINTS describes all constraint definitions on tables owned by the current user. Its columns are the same as those in "ALL_CONSTRAINTS".

A complete list of data dictionary views is available at the below address:

http://www.oracle.com/pls/tahiti/tahiti.catalog_views

Data Dictionary & Views

The views that summarize and display the information stored in the base tables of the data dictionary. These views decode the base table data into useful information. Most users are given access to the views rather than the base tables. All data dictionary information is stored in tables, but much of the data is presented to users through views. In other words, user don't get direct access to the tables of the data dictionary, they instead get access to the views, which provide somewhat limited access in order to protect the integrity of the data dictionary. Code of the View can be retrieved by using Data Dictionary using following syntax:

```
SELECT * FROM user_views;
```

VU-Gateway

Module 21: Introduction to Sequence and Synonyms with implementation in Oracle 11g

A. Motivation for sequence

Sequence is a feature supported by some database systems to produce unique auto number values on demand. Through sequences unique numbers can be generated automatically to be used in tables. A sequence is a database object from which multiple users can generate unique integers. Using a sequence generator to provide the value for a primary key in a table is an easy way to guarantee that the key value is unique. Sequence numbers are generated independently of tables, so the same sequence can be used for one or for multiple tables.

NNNNNNNN. Syntax of Sequence

The syntax of sequence is as follows:

```
CREATE SEQUENCE sequence_name
MINVALUE value
MAXVALUE value
START WITH value
INCREMENT BY value
CACHE value;
CYCLE / No CYCLE;
```

Where:

- MINVALUE: Specify the minimum value of the sequence.
- MAXVALUE: Specify the maximum value the sequence can generate.
- START WITH: Specify the first sequence number to be generated.
- INCREMENT BY: Specify the interval between sequence numbers
- CACHE: Specify how many values of the sequence the database pre-allocates and keeps in memory for faster access
- CYCLE: Specify CYCLE to indicate that the sequence continues to generate values after reaching either its maximum or minimum value. After an ascending sequence reaches its maximum value, it generates its minimum value. After a descending sequence reaches its minimum, it generates its maximum value
- NO CYCLE: Specify NOCYCLE to indicate that the sequence cannot generate more values after reaching its maximum or minimum value. This is the default.

OOOOOOOO. Implementing Sequence:

The following statement creates the sequence customers_seq in the sample schema. This sequence could be used to provide customer ID numbers when rows are added to the customers table.

© Copyright Virtual University of Pakistan

```
CREATE SEQUENCE customers_seq
START WITH     1000
INCREMENT BY   1
NOCACHE
NOCYCLE;
```

The first reference to customers_seq.nextval returns 1000. The second returns 1001. Each subsequent reference will return a value 1 greater than the previous reference.

PPPPPPPP. Sequence & Data Dictionary

```
SELECT * FROM user_sequences
WHERE sequence_name = 'customers_seq';
```

Retrieving value of sequence without using Data Dictionary

```
SELECT customers_seq.nextval, customers_seq.currval from dual;
```

QQQQQQQQ. Sequence & DML

Sequence numbers are generated independently of tables, so the same sequence can be used for one or for multiple tables.Sequence numbers are generated independently of tables, so the same sequence can be used for one or for multiple tables. Data Manipulation Language (DML) commands can be used with sequence for inserting, deleting or updating data in the sequence. Here is the example:

Consider an employee database in which sequence is created with the name of mySeq to auto generate employee numbers. Statements are given below for DML commands:

To insert a row into the sequence:

```
INSERT into emp (empno) VALUES (mySeq.nextVal);
```

To drop the sequence:

```
DROP sequence mySeq;
```

Module 22: Indexes in Databases

A. Index Basics

An index is a data structure that the database uses to find records within a table more quickly. Indexes are built on one or more columns of a table; each index maintains a list of values within that field that are sorted in ascending or descending order. Rather than sorting records on the field or fields during query execution, the system can simply access the rows in order of the index. Index, in simple terms, is a database object which helps in efficient searching of the data. Every time database table is accessed, all the rows in the table are searched to find the required data which limits the database performance. Indexes are used to quickly locate data without having to search every row in a database table every time a database table is accessed.

An example to clarify the concept, consider the data shown in the table. In order to search the salary of Aqeel, for example, all the rows in the table will be searched to locate the salary of Aqeel. In large tables, this can tremendously increase the run time of a query. Imagine the same scenario if the data is to be located from different tables.

Searching Without Indexes

Name	Salary
Ahmed	10000
Kamran	23000
Allen	42000
Samar	43000
Asim	42993
Zahid	51000
Gates	89000
Farhan	32999
Aqeel	39200

RRRRRRRR. Searching Using Index

The index is created on one or more columns. An index is created to make it easier for locating records in a table. The index will create another table or index structure, which is not a part of the actual table. (See the tables below from the previous example to understand better)

Actual Table

Searching Without Indexes

Name	Salary
Ahmed	10000
Kamran	23000
Allen	42000
Samar	43000
Asim	42993
Zahid	51000
Gates	89000
Farhan	32999
Aqeel	39200

Index Created Table

Searching With Indexes

Name	RowID
Ahmed	3948ABCF
Allen	3818AACF
Aqeel	3948ABCD
Asim	3948ABCA
Farhan	3248ABC0
Gates	3548ABCF
Kamran	3748ABCF
Samar	3248ABCF
Zahid	3148ABCF

Note that the data is sorted alphabetically in the index table

© Copyright Virtual University of Pakistan

It is like a supplement to the table and it will contain just 2 columns- the key values and a row ID/address to the actual row in the table. The index table / structure sort the data of the column in alphabetical order and the address or row ID in another column. As shown in the above tables, after the index table is created, the data has been sorted alphabetically and the row ID against all the records is also stored in that table. Now to search any record, index table can tell the row ID for each record.

SSSSSSSS. Creating Index in Oracle

Creating an index involves the CREATE INDEX statement, which allows you to name the index, to specify the table and which column or columns to index. The basic syntax is as follows:

```
CREATE INDEX index_name
ON table_name (column_name);
```

Examples are as follows:

```
CREATE INDEX test_emp
ON emp (ename);
```

```
CREATE INDEX emp_ind
ON emp (job, sal);
```

TTTTTTTT. Indexes and Data Dictionary

Index is also a Database Object and Data Dictionary is maintained. Following are the different Data dictionaries maintained for Indexes.

It will display all the indexes created by currently logged in user schema

```
SELECT * FROM user_indexes;
```

It will display all the columns of indexes created by currently logged in user schema

```
SELECT * FROM user_IND_COLUMNS;
```

© Copyright Virtual University of Pakistan

Module 23: Transaction

A. Transaction Basics

A transaction is a sequence of operations performed as a single logical unit of work, whether in a manual fashion by a user or automatically by some sort of a database program. A transaction is the propagation of one or more changes to the database. Practically, many SQL queries (DDL or DML)can be clubbed into a group and can be executed together as a part of a transaction. A logical unit of work i.e. Transaction must exhibit four properties, called the atomicity, consistency, isolation, and durability (ACID) properties, to qualify as a transaction. The details are given in the following.

UUUUUUUU. Atomicity Property of Transaction

This property states that a transaction must be treated as an atomic unit, that is, either all of its operations are executed or none. There must be no state in a database where a transaction is left partially completed. Atomicity ensures that all operations within the work unit are completed successfully; otherwise, the transaction is aborted at the point of failure, and previous operations are rolled back to their former state. Consider the following example:

- Transaction 1 (T1): Update command to update 100 rows
- Transaction 2 (T2): 20 rows are updated
- Transaction 3 (T3): System failure
- *Result: Database should rollback updates of 20 Rows*

XL. Consistency Property of Transaction

This property states that before and after transaction database should be in consistent state and System constraints should not be violated in any case. No transaction should have any adverse effect on the data residing in the database. If the database was in a consistent state before the execution of a transaction, it must remain consistent after the execution of the transaction as well. Example is as follows:

- Constraint: Only characters are allowed in ENAME column
- Transaction:
  - Start of Transaction
  - Commands
  - End of Transactions
- Results: There are values in ENAME column with _ and - character in it
- Summary: Transaction is not consistent because constraint is violated.

In the above example, during the transaction, a record is entered in the ENAME column with characters _ mentioned in it. This will result in inconsistency as the constraint is violated.

VU Gateway

This property states that all the steps during the execution of transactions are done without interference. This simply means that now two transactions can interfere with each other and if another transaction wants to access same data then it should wait. n a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system. No transaction will affect the existence of any other transaction.

LXXX. Durability

This property states that System should be capable enough to hold transactional data and if transaction fails then there should be backup and recovery process. The database should be durable enough to hold all its latest updates even if the system fails or restarts. After a transaction has completed, its effects are permanently in place in the system. The modifications persist even in the event of a system failure.

VU-Gateway

Module 24: Locks & Granularity

A. Concurrent Transaction

Concurrent execution of database transactions in a multi-user system means that any number of users can use the same database at the same time. Data is shared and accessed by multiple users simultaneously and this simultaneous access and processing of data may lead to dirty data. The uncontrolled execution of concurrent transactions in a multi-user environment can lead to various problems. The two main problems and examples of how they can occur are listed below.

YYYYYYY. Lost Update Problem

This problem occurs when multiple transactions are accessing same data have their operations interrupted in such a way, that one transaction will access the data before the other has applied any updates. In such scenario, the second transaction will not have access to the updated data i.e. the data updated after the first transaction. The isolation property of the transaction is violated as the transaction have interfered each other. Besides the transactions were having the access to the partial results only which results in 'dirty data'. Let's take an example:

Consider two transactions to book seat(s) in a flight and the system is processing two transactions in parallel. You can see that the operations are interleaved in such a way that Transaction 2 had access to the data before Transaction 1 could reduce the seats by 1. Transaction 2's operation of reducing the seats by 2 has been overwritten, resulting in an incorrect value of available seats.

Transaction1	Transaction2	Operation	Data Value
Read Flight Information			seats = 15
	Read Flight Information		seats = 15
	Book 2 seats	seats = seats -2	
Book 1 seat		seats = seats -1	
	Write seats		seats = 13
Write seats			seats = 14

In the end multiple transitions are having different value of one share variable which is WRONG, for Transaction1 value of Seats variable is 14 and for Transaction2 is 13 but seat will have value of 14 so value which transation2 is holding is wrong

ZZZZZZZZ.

This problem occurs when one transaction updates a data item, but has not yet committed the data permanently to the database. Because of failure, the transaction is rolled back and the data item is returned to its previous value. A second transaction accesses the updated data item before it is returned to its original value. The second transaction will read the share data between failure and rollback i.e. before the changes become permanent. Again, isolation property of the transaction is violated here. This is further illustrated in the following example:

© Copyright Virtual University of Pakistan

Again the example of two transactions to book seats in a flight. In this example, you can see that Transaction 2 has access to the updated data of Transaction 1 before the changes were committed permanently to the database. The transaction 1 rollbacks and this result in the dirty data as the incorrect value of the seats available will be shown.

Transaction1	Transaction2	Operation	Data Value
Read Flight Information			seats = 15
Book 1 seat		seats = seats -1	
Write seats			seats = 14 (Uncommitted)
	Read Flight Information		sea 14
	Book 2 seats	seats = seats -2	seats =
	Write seats		seats = 12
Rollback			seats = 15

In the end multiple transitions are having different value of one share variable which is WRONG, for Transaction1 value of Seats variable is 12 and for Transaction2 is 15 but seat will have value of 15 so value which transation2 is holding is wrong

AAAAAAAAA. Rationale for Lock

The issues in the concurrent transaction are due the isolation property of the transaction. The issues in concurrency may lead to the dirty or unreliable data. So this pinpoints a need for a mechanism to ensure the isolation property of all the transaction. Here the locks come in as a solution.

BBBBBBBBB. Lock Basics

A lock, as a read lock or write lock, is used when multiple users need to access a database concurrently. Locks avoid multiple users to access the share data on the same time and if one transaction is in progress then other transactions must wait to access same share data. This prevents data from being corrupted or invalidated when multiple users try to read while others write to the database. A read lock can be used to prevent other users from reading a record (or page) which is being updated.

CCCCCCCCC. Granularity of Locks

The granularity of locks in a database refers to how much of the data is locked at one time. In theory, a database server can lock as much as the entire database or as little as one column of data. Such extremes affect the concurrency (number of users that can access the data). The following table describes the multiple levels of locks.

Lock Level	Description
Database	• Only One Session can be created with Database • Not feasible to keep all the user in wait state • Feasible if there is major support update i-e to update Database to new version
File	File can be a single table, part of table or combination of multiple table
Table	• Entire Table is locked for particular user • Useful when change is impacting whole table.
Row	A row level lock applies to a row in a table. This is also the most commonly locking

	level, and practically all major database vendors support row level locks.
Column	Particular column or multiple columns are locked for a particular users.

DDDDDDDDD. Level of Locks

There are two levels of locks in database. These are:

1. ***Exclusive Lock:*** When a statement modifies data, its transaction holds an exclusive lock on data that prevents other transactions from accessing the data. This lock remains in place until the transaction holding the lock issues a commit or rollback. Table-level locking lowers concurrency in a multi-user system.
2. ***Shared Lock:*** When a statement reads data without making any modifications, its transaction obtains a shared lock on the data. Another transaction that tries to read the same data is permitted to read, but a transaction that tries to update the data will be prevented from doing so until the shared lock is released

Using exclusive locks the locked data can be read or processed by one user only. A request for another exclusive lock or for a shared lock is rejected, as shown in the 'Compatibility Graph'.

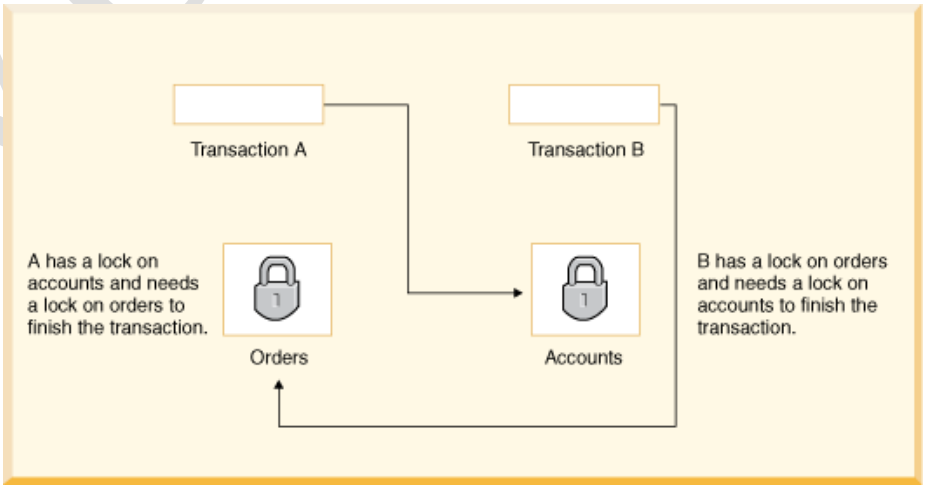
	S	X
S	true	false
X	false	false

Compatibility Graph

In shared locks, several users can read the same data at the same time, but as soon as a user edits the data, a second user can no longer access this data. Requests for further shared locks are accepted, even if they are issued by different users, but exclusive locks are rejected.

EEEEEEEEE. Deadlock

In a database, a deadlock is a situation in which two or more transactions are waiting for one another to give up locks. For example, Transaction A might hold a lock on some rows in the ACCOUNTS table and needs to update some rows in the ORDERS table to finish. Transaction B holds locks on those very rows in the ORDERS table but needs to update the rows in the ACCOUNTS table held by Transaction A. Transaction A cannot complete its transaction



because of the lock on Orders. Transaction B cannot complete its transaction because of the lock on Accounts. All activity comes to a halt and remains at a standstill forever unless the DBMS detects the deadlock and aborts one of the transactions.

FFFFFFFF. Deadlock Prevention

Database use to avoid deadlocks increase transaction throughput and reduce system overhead. To avoid deadlocks, each lock request is inspected to identify the risk of potential deadlock. If this inspection reveals the deadlock potential, the lock is not granted. Well, very few modern DBMS's can actually prevent or avoid deadlocks, because there's a lot of overhead required in order to do so. This is because the DBMS's that do try to prevent deadlocks have to try to predict what a database user will do next. Thus, every lock request must be accepted after assessing the deadlock potential of the lock request.

GGGGGGGGG. Deadlock Detection

Instead of deadlock prevention, the more popular approach to dealing with database deadlocks is deadlock detection. There are two common approaches to deadlock detection:

1. Set the lock wait (time for lock to be released) time period to a certain preset limit (like 5 seconds). So, if a session waits more than 5 seconds for a lock to free up, then that session will be terminated.
2. Inspect all the locks currently in place to see if there are any two sessions that have locked each other out and are in a state of deadlock.

In either of the above mentioned approaches, one of the requests will have to be terminated to stop the deadlock and any transaction changes which came before the request will have to be rolled back so that the other request can make progress and finish.

VU-Gateway